AeroGate: Explainable, Re-optimising Gate Assignment under Irregular Operations at Toronto Pearson

Shawn Wei
University of Waterloo
Brock Solutions Airport Innovation Challenge, subproblem 03 (gate assignment)

November 2026

Abstract

Irregular operations (delays, aircraft swaps, cancellations, gate outages and diversions) force gate planners to re-plan many times a day, under hard safety rules and with little time to check a proposal. We present *AeroGate*, a rolling-horizon gate assignment system that re-solves the whole gate plan at every information update with an exact constraint-programming model whose objective is the evaluator's own score. Three ingredients make the plans usable in operations: (i) *move-ordering constraints* that guarantee a plan can be executed one reassignment at a time, (ii) a *safety replay* that re-checks every action with the evaluator's rules, inside a chain of fallbacks so that the decision function never fails, and (iii) a *per-decision explanation* that records the rules each choice passed, its cost, and why every other gate lost. A realism layer for Terminal 1 at Toronto Pearson (CYYZ), built from OpenStreetMap, adds US pre-clearance and swing gates with security sweeps, wingtip clearance between neighbouring stands, tarmac holds priced against Canada's three-hour APPR limit, and connecting passengers. On the 12 challenge scenarios AeroGate has no hard failures and the lowest total score (96.2 against 104.3 and 96.8 for the provided greedy and first-fit planners); on 40 randomly generated days it fails once (an unavoidable case) against 3 and 6, and wins 26 of the 33 days that all planners complete. On a Pearson evening bank it gates all 83 flights, and every re-plan after the first is proven optimal in at most 1.5 s. A surface simulation animates the plan on the real airfield (dual arrival runways, runway crossings, conflict-free taxiing and taxiway holding) so that operators can see, not just read, the plan.

1 Introduction

Gate assignment decides, for every flight, the gate (stand) it uses while on the ground. In normal operations the schedule is planned the day before; on the day, irregular operations (IROPS) keep invalidating it. A 40-minute delay overlaps the next aircraft at the gate; an equipment swap brings a widebody that needs a jetbridge and a bigger stand; a jetbridge fails; a diverted aircraft arrives with no booking. Each event triggers a re-plan, and each re-plan must (a) never break a hard rule, (b) move as few published gates as possible, because moves cost passengers and staff, and (c) be trusted by the person who has to approve it within minutes.

The challenge environment (the *GMS evaluator*) replays a day as a sequence of updates. At each update the solution receives the current state and returns new assignments and reassignments, which the evaluator applies one at a time, rejecting the run if any action breaks a hard rule. The score penalises walking distance, premium-gate use, reassignments and unassigned flights.

Contributions.

1. An exact rolling-horizon CP-SAT model whose objective is the evaluator's score, re-solved over all flights at every update (Section 4).
2. Move-ordering constraints with integer positions that make every optimal plan executable under sequential application, and a no-good loop for residual cycles (Section 4.4).
3. A never-fail decision function: safety replay with the evaluator's own checks and a chain of fallbacks (Section 5).
4. Counterfactual explanations for every decision (Section 6).
5. A Pearson realism layer from OpenStreetMap: pre-clearance, swing gates, wingtip clearance, tarmac holds under APPR, connections (Section 7).
6. A surface simulation and display: arrival sequencing on two runways with crossings, rule-based conflict-free taxiing, holding on unused taxiways, and overlap-free rendering (Section 8).

2 Related work

Gate assignment is a well-studied variant of interval scheduling and assignment; Dorndorf et al. [1] and Bouras et al. [2] survey models and objectives (walking distance, robustness, towing, and disruption recovery). Over-constrained instances, where not every flight can be gated, are treated by Ding et al. [3]; multi-objective formulations by Yan and Huo [4]. Most work optimises a static plan. Our setting is a *rolling* one with an external executor that applies actions sequentially and fails hard, which motivates the ordering constraints and safety replay. We use the CP-SAT solver of OR-Tools [5, 6], whose optional interval variables and no-overlap constraints express gate occupancy directly. Separation rules follow ICAO PANS-ATM [7]; tarmac delay rules follow Canada's Air Passenger Protection Regulations [8].

3 Problem at one update

Time is measured in integer minutes; intervals are half-open. At an update at time t the observation gives the following.

Gates. A set $\mathcal{G}$; each $g \in \mathcal{G}$ has the flight categories it may serve Z_g, maximum wingspan W_g, jetbridge flag J_g, walking distance d_g (metres), outage windows Ω_g, a position $\mathbf{p}_g \in \mathbb{R}^2$ in terminal metres, and (Pearson) a security-sweep time σ_g.

Flights. A set $\mathcal{F}$ of decision flights; each f has category c_f, wingspan w_f, jetbridge need j_f, presence intervals $I_f = \{[s, e)\}$ at the airport, a current gate γ_f (possibly none), whether its presence has begun (a_f), whether the airport caused its latest change (ϕ_f, e.g. a delay or swap), a priority flag π_f^{prio} (diversion, medical), and (Pearson) minutes already waited on the tarmac τ_f. Flights whose gate can no longer change are *fixed* and block their gate.

Hard rules. Flight f may use gate g only if

$$c_f \in Z_g, \qquad w_f \le W_g, \qquad j_f \Rightarrow J_g, \tag{1}$$

no interval of f meets an outage of g, and no two aircraft overlap at a gate: $[s_1, e_1)$ and $[s_2, e_2)$ conflict iff $s_1 < e_2 + \beta$ and $s_2 < e_1 + \beta$ for turnaround buffer β (0 in the challenge).

Score. The evaluator charges, over the final plan,

$$S = \sum_{f} \Big(0.01\, d_{g(f)} + 5\, \mathbf{1}[\text{domestic at intl gate}] \Big) + 10\, M_{\text{before}} + 50\, M_{\text{after}} + 100\, U + 300\, U_{\text{prio}}, \quad (2)$$

where M_{before} and M_{after} count reassignments chosen by the solution before and after the aircraft's presence began, and U, U_{prio} count ungated regular and priority flights. Moves forced by the airport (a flagged flight whose old gate became invalid) are not charged. Any hard-rule violation ends the run, so it outweighs any soft saving.

4 Optimisation model

Figure 1 shows the pipeline run at every update; this section describes the model at its centre, and Sections 5 and 6 the steps around it.

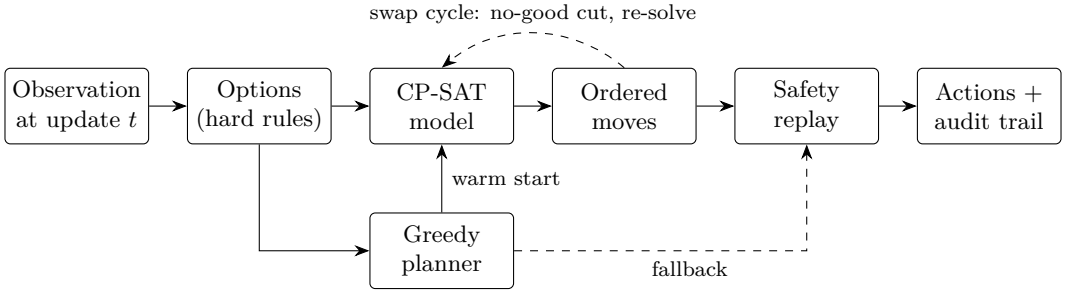


Figure 1: Decision pipeline at one update. Options already satisfy the hard rules; CP-SAT optimises the evaluator's score with a greedy warm start; moves are ordered so they can be applied one at a time; every action is re-checked with the evaluator's own rules before it is returned, with the greedy plan (and then the provided reference planner) as fallbacks.

4.1 Options

Rather than pairing flights and gates inside the solver, we enumerate *options* that already satisfy the hard rules. For flight f, gate g satisfying (1) and tarmac hold $h \in H_f$ (Pearson only; $H_f = \{0\}$ otherwise), the option $o = (f, g, h)$ has shifted presence

$$I_o = \{[s_1 + h,\ \max(e_1,\ s_1 + h + m_f))\} \cup \{I_f \setminus [s_1, e_1)\}, \quad (3)$$

where $[s_1, e_1)$ is the first interval and $m_f = 30$ the minimum turn. Options that hit an outage or a fixed occupancy are discarded and recorded (with the reason) for the explanation. Its occupancy on g is $P_o = \text{merge}\{[s, \max(e + \beta, s + 1))\}$; a flight that stays on its gate blocks the union of its old and new intervals. Every flight that holds a gate also gets a last-resort option "stay, although invalid" with cost 10^7, so the model is always feasible.

4.2 Variables and constraints

Binary x_o selects option $o \in \mathcal{O}_f$; binary u_f marks flight f ungated.

$$\sum_{o \in \mathcal{O}_f} x_o = 1 \qquad f \in \mathcal{F},\ \gamma_f \neq \varnothing \tag{4}$$

$$\sum_{o \in \mathcal{O}_f} x_o + u_f = 1 \qquad f \in \mathcal{F},\ \gamma_f = \varnothing \tag{5}$$

$$\textsc{NoOverlap}(\{\langle P_o, x_o \rangle : o \text{ on } g\}) \qquad g \in \mathcal{G} \tag{6}$$

$$x_a + x_b \leq 1 \qquad (a,b) \in \mathcal{C}_{\text{sweep}} \cup \mathcal{C}_{\text{wing}} \tag{7}$$

Constraint (4) is an equality because the evaluator has no "unassign" action: a flight that holds a gate must end on some gate. Constraint (6) uses optional fixed-size interval variables, present iff $x_o = 1$. The pair sets in (7) are

$$\mathcal{C}_{\text{sweep}} = \{(a,b) : g_a = g_b = g,\ |Z_g| \geq 2,\ c_a \neq c_b,\ s_b \in [e_a, e_a + \sigma_g)\},$$
$$\mathcal{C}_{\text{wing}} = \{(a,b) : (g_a, g_b, L) \in \mathcal{A},\ I_a \cap I_b \neq \varnothing,\ w_a + w_b > L\},$$

where $\mathcal{A}$ lists neighbouring gate pairs with a combined-wingspan limit L (Section 7). Both are found with a sweep over intervals sorted by start time.

4.3 Objective

All costs are integers (evaluator points $\times 100$). The cost of option $o = (f, g, h)$ is

$$c_o = d_g + 500\,\mathbf{1}[\text{premium}] + 1000\,\mathbf{1}[\text{move}, \neg a_f, \neg \phi_f] + 5000\,\mathbf{1}[\text{move}, a_f] + \kappa_f\,\Phi(\tau_f + h), \tag{8}$$

with $\kappa_f = 3$ for priority flights and 1 otherwise, and the convex tarmac-hold penalty

$$\Phi(t) = 20\,t + 0.9\,U_0\,(t/180)^4, \qquad U_0 = 10\,000, \tag{9}$$

which is cheap for short holds and climbs steeply toward the 180-minute APPR limit (options beyond it are not generated). An ungated flight costs $U_f = 10\,000$ ($30\,000$ if priority) plus $\Phi(\tau_f)$ if it is already waiting. Connecting passengers (a, d, n) add a walking term: integer positions X_f, Y_f are tied to the chosen gate by $x_o \Rightarrow (X_f, Y_f) = \mathbf{p}_{g_o}$, and

$$\min \sum_o c_o x_o + \sum_f U_f u_f + \sum_{(a,d,n)} \lceil \rho n \rceil \,(|X_a - X_d| + |Y_a - Y_d|), \tag{10}$$

with the absolute values linearised by CP-SAT's `AddAbsEquality`. Without the Pearson-only tarmac-hold and connection terms, (10) equals $100\,S$ in (2) on the decision flights, so the solver only moves a published gate when the move saves more than it costs. Weights are read live from the evaluator's configuration, so a re-tuned evaluator is followed automatically.

4.4 Move ordering

The evaluator applies reassignments one by one, each checked against the gates *as they are at that moment*. A plan that swaps two flights (A to B's gate, B to A's gate) is optimal but cannot be applied. Let $\mathcal{M} = \{f : \gamma_f \neq \varnothing\}$ and introduce integer positions $\pi_f \in \{0, \ldots, |\mathcal{M}|\}$. For every option o that moves f onto a gate currently held by $b \neq f$ with overlapping intervals,

$$x_o \Rightarrow \pi_b + 1 \leq \pi_f. \tag{11}$$

Proposition 1. *In any solution of* (4)–(11)*, applying the reassignments in increasing order of* π *never moves a flight onto a gate still held, at an overlapping time, by a flight that has not moved yet.*

Proof. Suppose f moves onto gate g held by b with overlapping intervals. Then (11) gives $\pi_b < \pi_f$, so b is processed first. If b stays on g, its option and f's option both occupy g at overlapping times, contradicting (6); so b moves away first. A cycle $f_1 \to \cdots \to f_k \to f_1$ of such dependencies would require $\pi_{f_1} < \pi_{f_2} < \cdots < \pi_{f_1}$, which is impossible. □

The occupancy the evaluator checks at application time can differ slightly from the model's (e.g. a flagged flight's old booking), so the action builder sorts the moves topologically and, if a residual cycle remains, adds the no-good $\bigvee_{(f,g)\in\text{cycle}} \neg x_{f,g}$ and re-solves (at most eight times, then falls back to the greedy plan).

4.5 Solving

CP-SAT runs with one search worker, a fixed random seed and a *deterministic* time budget (2.0 deterministic seconds per update; 2.5 in the Pearson layer), with the wall clock only as a safety net. The same input therefore yields the same plan on every run, which matters for explanations and for audit. The greedy plan of Algorithm 1 is passed as a solution hint (warm start) and is also the fallback if OR-Tools is missing or fails.

Algorithm 1 Greedy planner (warm start and fallback)

$B \leftarrow$ empty board (checks overlap, sweep and wingtip rules incrementally)
for f with $\gamma_f \neq \varnothing$ **do**
    place the stay option of f on B if it exists
end for
for flagged f in order of first presence **do**
    remove f from B; if its stay option still fits, keep it
    otherwise place the cheapest option of f that fits (or the last-resort option)
end for
sort waiting flights by (priority first, longest tarmac wait, fewest options, earliest)
for each waiting f **do**
    place the cheapest option that fits, or leave f ungated
end for

5 Execution safety

A hard failure ends an evaluator run, so the decision function must not fail in any way. Algorithm 2 shows the per-update pipeline.

Replay simulates the evaluator's state transitions with its own compatibility code and drops any action that would be rejected, so the output contains only actions the evaluator accepts. Combined with the fallbacks, the function always returns and never returns an invalid action.

6 Explanations

Operators approve plans they can check. For every decision AeroGate records (a) the rules the chosen option passed, with the numbers ("wingspan 60 m within gate limit 65 m", "wingtip clearance

Algorithm 2 Decision at one update

```
function DECIDE(observation)
    P ← problem from the observation (weights read from the evaluator)
    repeat                                                    ▷ at most 8 rounds
        plan ← CP-SAT with cuts (greedy hint); actions ← ordered moves
        if a residual swap cycle exists, add a no-good cut
    until no cycle
    kept, errors ← REPLAY(actions)     ▷ the evaluator's own compatibility and overlap checks
    if errors then
        use the greedy plan instead if its replay has fewer errors
    end if
    record the explanation (never allowed to raise)
    return kept
end function
on any exception: the provided reference planner, replayed; on a further exception: no actions
```

with F62: combined 96 m $\leq$ 100 m"); (b) the cost breakdown in evaluator points; and (c) a verdict for *every other gate*, by evaluating the counterfactual option $o' = (f, g')$ against the final plan:

- **rule**: o' was never generated; the failed rule of (1), an outage, or a fixed occupancy is quoted;
- **busy**: o' is legal but conflicts in the final plan with a named flight (overlap (6), sweep or wingtip pair (7));
- **costlier**: o' is feasible but $\Delta = c_{o'} - c_o > 0$; Δ and its main driver (e.g. "+0.90 pts: longer walk, 490 m vs 400 m") are reported.

The dashboard shows these per flight, colours the gates on the map accordingly, and summarises each update in plain language ("moved D51/F51 to D39 because of a delay; re-planned before landing, so no charge"); Figure 2 shows one decision.

7 Pearson realism layer

The challenge data knows only cargo, domestic and international. We built a second model of Terminal 1 at Toronto Pearson (30 contact gates and two Cargo West stands) from an OpenStreetMap extract [10].

Geometry. Gate names (including dual-named swing gates such as E67/F67), stand centrelines and nose-in headings come from OSM. A stand is widebody-capable if its centreline is at least 100 m long or it has MARS sub-stands. Walking distance is $d_g = 1.3\,\|\mathbf{p}_g - \mathbf{c}\|$ from the terminal centre $\mathbf{c}$, rounded to 10 m.

Wingtip clearance. Two aircraft side by side need half their spans plus a clearance margin between centrelines: $(w_1 + w_2)/2 + 7.5 \leq \delta$. Probing each stand 40 m from the nose and measuring the spacing $\delta < 75$ m to its neighbour gives the combined-span limit

$$L = \max\big(76,\ \mathrm{round}(2(\delta - 7.5))\big), \tag{12}$$

kept when $L < 130$ m (24 pairs). The floor of 76 m keeps two narrowbodies abreast legal where OSM lines converge at pier corners.

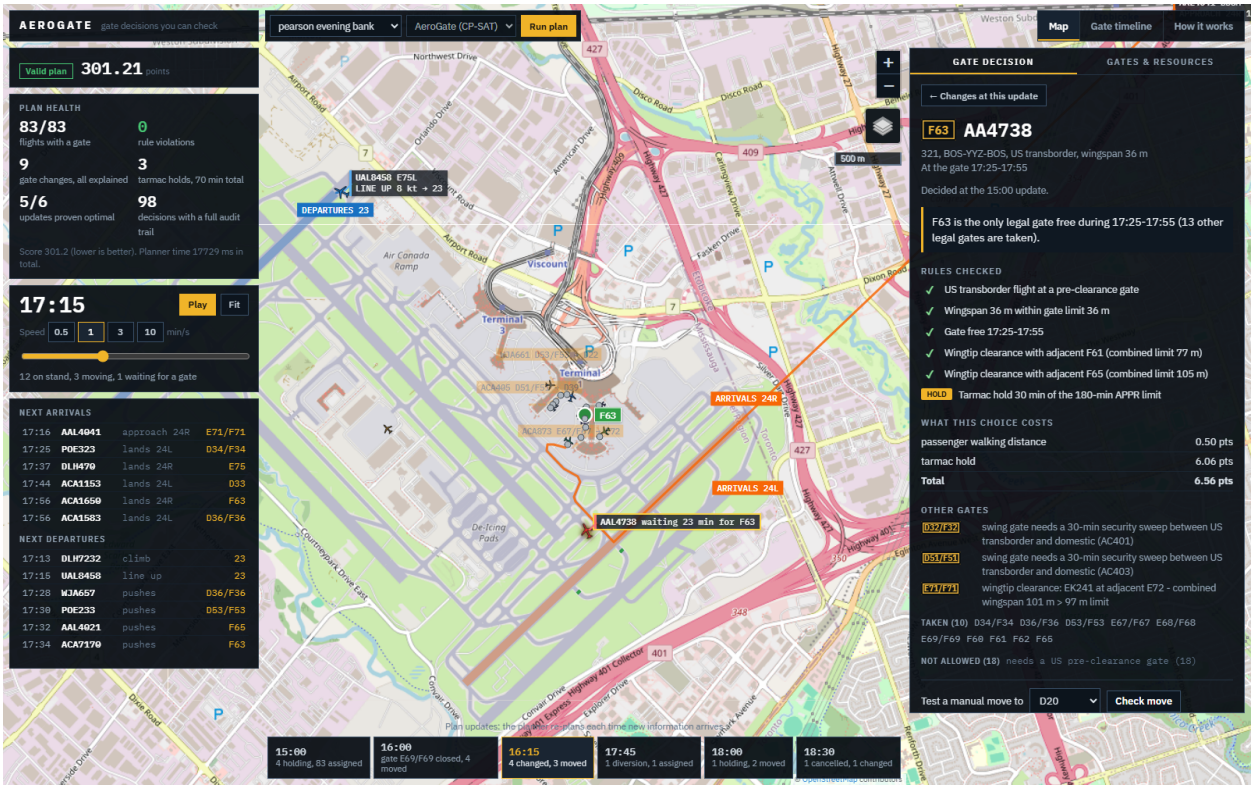


Figure 2: The operator dashboard on the Pearson evening bank at 17:15. AA4738 (US transborder A321) holds on a taxiway holding spot for its only free legal gate, F63; its route to the stand is drawn. The right panel is the decision record: the rules F63 passed with their numbers, the cost in evaluator points (walking 0.50, tarmac hold 6.06), the alternatives blocked by swing-gate sweeps or wingtip clearance, and one-line summaries of the gates that were taken or not allowed.

Operational rules. US transborder flights need pre-clearance gates (Pier F and the D/F, E/F swing gates); a swing gate needs a $\sigma_g = 30$-minute security sweep between flights of different zones; an arrival with no free gate may hold on the tarmac for $h \in \{0, 10, 20, 30, 60, 90, 120, 180\}$ minutes priced by (9), three times higher for medical diversions; connecting passengers add 0.02 points per passenger per 100 m of Manhattan distance between the two gates.

Scenario. An evening bank (83 flights, six updates from 15:00 to 18:30) with a jetbridge failure on swing gate E69/F69, a thunderstorm bunching seven arrivals, a 777 medical diversion, a 787 swap on a US flight and a cancellation.

8 Surface simulation and display

The optimiser decides gates and times; the dashboard animates the result on the real airfield so operators can see congestion and conflicts. The simulation is for validation and display only; it does not feed back into the plan.

8.1 Runways and arrival sequencing

Pearson's usual west-flow configuration is used: arrivals on 24R and 24L, departures from 23. Taxi routes are shortest paths (Dijkstra [9]) on the OSM taxiway graph (3,251 nodes, 3,682 edges) with runway edges weighted $\times 12$, so routes cross runways but never use them as taxiways. For each gate and landing runway r the exit is the junction 1.1–2.7 km down the runway that minimises taxi length ℓ_r. The plan's gate-in time G_f gives a touchdown target per runway

$$t^0_{f,r} = G_f - h_f - R - \ell_r / v_{\text{taxi}} - \ell_{\text{entry}} / v_{\text{stand}}, \tag{13}$$

with roll-out $R = 1.1$ min, $v_{\text{taxi}} = 480$ m/min and $v_{\text{stand}} = 110$ m/min. Arrivals are taken in order of $\min_r t^0_{f,r}$ and each is given the runway minimising its lateness $t_{f,r} - t^0_{f,r}$ (ties: the runway idle longest), where

$$t_{f,r} = \max\Big(t^0_{f,r},\ t_{\text{prev}(r)} + \max\big(\tfrac{60\,s(\text{prev},f)}{140},\ 1.4,\ X_r\big),\ t_{\text{last}} + 0.65\Big). \tag{14}$$

Here $s \in \{3, 4, 5\}$ NM is the wake separation (3 behind a medium, 4 for a heavy behind a heavy, 5 for a medium behind a heavy) flown at 140 kt, 0.65 min staggers landings on the two close parallels (about 1.5 NM diagonal), and $X_r = 2.6$ min on a runway that other traffic must cross, else 0.

Runway crossings. The 24L arrivals reach Terminal 1 by crossing 24R at the same two junctions where 24R arrivals exit. A crossing is allowed at time T only if no 24R arrival has $T > t - 0.8$ (within about 2 NM of touchdown) without having already turned off and moved 120 m clear. A landing therefore blocks crossings for roughly $0.8 + 1.1 + 0.25 \approx 2.15$ min; spacing 24R landings at least $X_r = 2.6$ min apart leaves a gap of at least 0.45 min after every landing, so crossing traffic cannot be starved.

8.2 Ground movement

Ground traffic is a time-stepped simulation with $\Delta t = 3$ s. Each aircraft follows its route and obeys rules designed so that every stop eventually clears:

1. stop 105 m behind an aircraft ahead in the same direction, looking 120 m ahead along the route so a merge onto another aircraft's taxiway is seen before the two are alongside;
2. at crossings and head-on, the aircraft that started later steps to a parallel lane 60 m to the side, choosing the side by where the two will be closest and never within 45 m of a parked aircraft; a stopped aircraft keeps its lane;
3. one pushback at a time per alley, only when no aircraft is about to pass behind the stand;
4. departures queue nose to tail at the runway 23 holding point and take off 1.5 min apart (2 min behind a heavy);
5. an arrival whose stand is still occupied waits where it is clear of that stand's taxi-out route, so the departing aircraft can get past.

Holding on unused taxiways. An aircraft that must wait for its gate (a planned tarmac hold, or a stand that is still occupied) should not stop on the busy apron. Holding spots are the taxiway nodes v with

$$\text{dist}(v, \text{routes}) \geq 80 \text{ m}, \quad \text{dist}(v, \text{stands}) \geq 80 \text{ m}, \quad \text{dist}(v, \text{runways}) \geq 150 \text{ m}, \tag{15}$$

on the terminal side of every runway, chosen greedily nearest the arrival flow with 150 m spacing (10 spots). The browser runs Dijkstra from each spot over the same graph. The simulation runs twice: the first pass finds the arrivals that would wait for their stand for 45 s or more; the second books each waiting aircraft onto the spot with the shortest detour $\ell(\text{exit} \to v) + \ell(v \to \text{stand})$ subject to: the spot is free for the hold interval (taken from the first pass, with margins), its route passes no spot occupied at that time (70 m), and the spot lies on no other booked route while it is in use.

8.3 Overlap-free rendering

Zoomed out, true-size aircraft are a few pixels wide; drawing them larger keeps them visible but makes neighbours overlap. Each aircraft i is drawn at scale $k_i \in [0.8, k_i^{\max}]$, where $k_i^{\max} = \max(1, \ell^{\min}/(\text{span}_i/m))$, m is metres per pixel and $\ell^{\min}$ is 17 px (12 px parked). The plane shape $A_i(k)$ is approximated by three oriented rectangles (fuselage, wing, tail). For each close pair, s_{ij} is the largest value on a ladder of scales such that $A_i(\min(s, k_i^{\max}))$ and $A_j(\min(s, k_j^{\max}))$ are disjoint for every ladder value up to s (tested with the separating-axis theorem), and

$$k_i = \min\left(k_i^{\max},\ \min_j s_{ij}\right), \tag{16}$$

followed by a repair pass that shrinks the larger icon of any pair that still touches. Neighbours that would touch even at true size (wingtips at corner stands) are drawn up to 20% smaller.

8.4 Ground crew and baggage (estimates, display only)

Per pier p, ramp-crew demand is $C_p(T) = \sum_f n_f \, \mathbf{1}[T \in W_f]$ with $n_f = 2$ for a widebody and 1 otherwise, where W_f covers 25 min after in-block (40 widebody) and 35 min before off-block (50 widebody) to 5 min after. Baggage load is $B_p(T) = \sum_f b_f/|V_f| \, \mathbf{1}[T \in V_f]$ bags per hour, with $b_f =$ seats $\times$ load factor $\times$ bags per passenger, unloaded over 45 min after in-block and made up over 150 min before off-block, against belts of 1,200 bags/h. Staffing (D 14, F 6, E 16, cargo 2 crews) and belts (D 2, F 1, E 4) are estimates; the planner does not use them.

9 Results

Table 1: Evaluator results (lower score is better). Generated days use random layouts of 6–40 gates and 12–90 flights with delays, swaps, cancellations, reinstatements, outages and diversions announced 30–180 min ahead.

	AeroGate	Provided greedy	Provided first-fit
12 challenge scenarios, total score	**96.2**	104.3	96.8
Hard failures, challenge scenarios	**0**	0	0
40 generated days, hard failures	**1**	3	6
Total score, 33 days all planners finish	**3,188**	3,565	5,783
Days won outright (of 33)	**26**	5	0

Benchmarks. Table 1 summarises the evaluator runs. Every generated day passes the same guarantee as the official scenarios: after each update all active flights can be placed at once. The one AeroGate failure (`gen_022`) cannot be avoided by any planner: an outage hits a parked widebody

while all seven compatible gates hold parked aircraft, and the evaluator cannot swap two parked aircraft.

Ablation. A robustness bonus for slack between consecutive turns (a tight-turn penalty linear in the gap within a 45-minute window) made the total on generated days worse (3,582 vs 3,078 at the time of the test): delays are announced before arrival, and the evaluator lets a delayed flight move for free then, so slack bought in advance is mostly wasted. It is switched off.

Table 2: Pearson evening bank: plan and per-update solve.

Update	CP-SAT status	Solve (ms)	Decisions	Gate moves
15:00 (initial plan)	feasible	9,327	83	0
16:00 gate E69/F69 closed	optimal	1,078	4	4
16:15 storm, delays	optimal	1,500	7	3
17:45 medical diversion	optimal	466	1	0
18:00 787 swap, hold	optimal	622	2	2
18:30 cancellation	optimal	372	1	0

Pearson. All 83 flights are gated with no rule violation; the score is 301.21 points (walking 223.30, three charged moves 30.00, connecting passengers 29.79, tarmac holds 18.12). Table 2 gives the per-update solve: the initial 83-flight plan uses the full deterministic budget, and every re-plan after it is proven optimal in at most 1.5 s.

Against greedy rules on the same evening. To isolate the value of optimisation we re-ran the Pearson evening with identical rules, options and disruptions, but with the greedy planner of Algorithm 1 alone: keep every gate, re-fit what breaks, give each waiting flight the first good free gate. This is how rule-based tools (and a busy planner) re-plan. Table 3 compares the two. Both gate every flight without a rule violation; the difference is the tarmac. Looking at the whole evening at once, AeroGate spends three charged gate moves to avoid holds the greedy rules cannot see coming, cutting held passenger-hours by 82% and the score by 20%.

Table 3: Pearson evening bank, same rules and disruptions: AeroGate against greedy rules (no optimisation). Passenger-hours assume an 85% load factor.

	AeroGate	Greedy rules
Flights with a gate	83/83	83/83
Rule violations	0	0
Flights held on the tarmac	**3**	11
Minutes held on the tarmac	**70**	320
Passenger-hours held on the tarmac	**234**	1,320
Gate changes	**9**	10
Score (lower is better)	**301.2**	377.9

Surface simulation. Over the whole evening, sampled every 6 s: 33 landings on 24R and 32 on 24L; all 76 departures from 23; no aircraft is ever on a runway while another lands on it; two 24L

arrivals hold short of 24R (at most 1.5 min); every aircraft waiting for a gate waits on a taxiway holding spot; the closest approach between two moving aircraft is 39 m, and drawn aircraft never overlap at 1, 3.5 or 4.9 metres per pixel.

10 Discussion and limitations

The exact model is small enough to re-solve the whole day at every update, which removes the usual trade-off between local repair and global quality: the solver sees every knock-on effect of a move. Equality with the evaluator's score makes the plan's cost auditable, and the explanation turns the solver's choice into statements an operator can verify.

Limitations. The evaluator has no "unassign" or remote-stand action, so some outage cases cannot be saved (a real system would tow to a remote stand; the Pearson layer can hold on the tarmac). Stand sizes, walking distances, crews and baggage figures in the Pearson layer are estimates. The surface simulation is rule-based and does not optimise taxi routes or runway sequences jointly with gates. The first plan of a large day uses the full single-thread budget.

Future work. Learning delay distributions from A-CDM milestones (TOBT/TSAT) to price buffers by probability; warm-starting from the previous update's plan; joint gate, ground-crew and baggage optimisation using the pier estimates as constraints; and solving per pier in parallel.

11 Conclusion

AeroGate treats gate re-planning as a sequence of exact optimisations whose objective is the score that judges the plan, whose output is guaranteed executable and valid, and whose every decision comes with its reasons. On the challenge benchmarks it is never worse on hard failures and best on total score, and on a realistic Pearson evening it keeps every flight gated while proving its re-plans optimal in about a second.

References

[1] U. Dorndorf, A. Drexl, Y. Nikulin, E. Pesch. Flight gate scheduling: State-of-the-art and recent developments. *Omega*, 35(3):326–334, 2007.

[2] A. Bouras, M. A. Ghaleb, U. S. Suryahatmaja, A. M. Salem. The airport gate assignment problem: A survey. *The Scientific World Journal*, 2014:923859, 2014.

[3] H. Ding, A. Lim, B. Rodrigues, Y. Zhu. The over-constrained airport gate assignment problem. *Computers & Operations Research*, 32(7):1867–1880, 2005.

[4] S. Yan, C.-M. Huo. Optimization of multiple objective gate assignments. *Transportation Research Part A*, 35(5):413–432, 2001.

[5] L. Perron, V. Furnon. OR-Tools. Google, `https://developers.google.com/optimization`.

[6] L. Perron, F. Didier, S. Gay. The CP-SAT-LP solver. In *Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs 280, 2023.

[7] ICAO. *Procedures for Air Navigation Services: Air Traffic Management* (Doc 4444).

[8] Canada. *Air Passenger Protection Regulations*, SOR/2019-150.

[9] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[10] OpenStreetMap contributors. Map data available under the Open Database License, `https://www.openstreetmap.org`.